

Secure Artifact Publishing Guide

A two repository implementation book for the Terraform platform and the brewery data job

Prepared from the current working trees on 2026-09-30. This guide is written for the person operating both repositories and assumes AWS account 930 628 638 871 in us-east-1.

The key decision in this edition: the code repository has only a main branch. A successful push to data-platform-jobs/main is therefore authorized to publish a tested build into the development artifact bucket. Infrastructure branches still map dev, staging, and main to their own Terraform environments.

What this book gives you

This is an implementation manual, not just a design note. It records the purpose of both repositories, explains the security boundary, shows the exact current contents of the relevant files, explains what each file is responsible for, and gives a sequenced path from the current artifact-publishing work to a Terraform-pinned AWS Glue job.

- An accurate explanation of the branch mismatch that prevented the first publishing run.
- The intended secure flow from data-platform-jobs/main to lead-de-dev-artifacts.
- A file-by-file reference appendix with the complete current contents of every tracked source, configuration, workflow, and fixture file in scope.
- A recommended reference implementation for the next Glue deployment phase, including its IAM responsibilities, artifact pinning, and operational checks.
- Known limitations and decisions that must be made before treating the learning platform as a production deployment.

How to use it

Read Chapters 1 through 6 before changing anything. Use Chapter 7 as the immediate execution checklist. The appendices are a source-of-truth snapshot for review and learning; do not blindly replace a file simply because it appears there. The future Glue reference is deliberately separated from the files already in the repositories so you can create a small reviewed change rather than introduce everything at once.

Contents

1. Executive summary and the operating model
2. Architecture and trust boundaries
3. Repository responsibilities and branch model
4. Secure artifact publishing implementation
5. What the current implementation guarantees and does not guarantee
6. Next platform milestone: a Glue job pinned to an artifact
7. Runbook from the current state
8. Troubleshooting and verification
9. Security hardening backlog
10. Appendix A. Infrastructure repository file reference
11. Appendix B. Data jobs repository file reference
12. Appendix C. Future Glue Terraform reference
13. Appendix D. Commands and source references

1. Executive summary and the operating model

Two repositories are intentionally collaborating. The infrastructure repository owns durable cloud infrastructure, AWS identity, and the configuration that turns a published artifact into an AWS Glue job. The data-platform-jobs repository owns Python source code, tests, packaging, and the CI run that proves a build is good enough to publish.

The immediate goal is not to give GitHub broad AWS access. It is to give one GitHub workflow a temporary identity with exactly enough power to write two build outputs to a single prefix in a single development bucket. AWS issues short-lived credentials only after GitHub proves the workflow came from the allowed repository and branch.

The next goal is a separate execution identity for AWS Glue. Glue should be able to read one selected artifact and operate on the data lake. It must not reuse the GitHub publisher role. Separating publisher and executor roles is the heart of the design: code delivery and data processing have different permissions and different failure modes.

The corrected branch decision

The original publishing example used a dev branch in the code repository. The actual data-platform-jobs repository has only main. The infrastructure repository has dev, staging, and main because it uses those branches as environment promotion tracks. Those are different branch models and they must not be conflated.

Repository	Branch event	Meaning	Target
data-platform-jobs	push to main	tested code release candidate	development artifact prefix
learn-terraform-get-started-aws	push to dev	apply development infrastructure	Terraform dev workspace
learn-terraform-get-started-aws	push to staging	apply staging infrastructure	Terraform staging workspace
learn-terraform-get-started-aws	push to main	apply production infrastructure	Terraform prod workspace

This means a change to the code workflow must use refs/heads/main, and the AWS role trust policy must name refs/heads/main. The role can still be called a dev publisher because its permissions point only at lead-de-dev-artifacts. The environment is determined by the AWS resource access, not by the source branch name.

Success criteria

- A pull request in data-platform-jobs lints, tests, builds, and uploads a temporary GitHub artifact, but cannot obtain AWS credentials or write S3 objects.
- A push to data-platform-jobs/main obtains a short-lived OIDC session and writes a wheel and glue_job.py only below lead-de-dev-artifacts/glue-artifacts/<Git SHA>/.
- A later Terraform change creates a Glue job whose script location and library location each contain one explicit Git SHA.
- A Glue runtime role, not the publisher role, reads the exact code artifact and writes only the required development data locations.

2. Architecture and trust boundaries

The architecture has four principals. Keeping them distinct prevents a convenience decision from turning into a broad security grant.

Principal	Identity	Allowed work	Must not do
GitHub test job on a pull request	GitHub Actions token without AWS OIDC use	lint, test, build, temporary artifact	write to AWS
GitHub publisher on main	OIDC session for lead-de-dev-jobs-artifact-publisher	write the selected artifacts to the dev Glue prefix	read data, delete data, deploy infrastructure
Terraform CI	currently stored AWS credentials	plan and apply the infrastructure repository	reuse the narrow publisher role
AWS Glue runtime	separate Glue service role	read pinned code and operate the data pipeline	publish code or modify infrastructure

End-to-end flow

```
Developer changes Python code
|
v
GitHub pull request: lint -> Spark/Delta tests -> wheel build -> temporary artifact
| (no AWS step; publishing conditions are false)
v
Merge into data-platform-jobs/main
|
v
GitHub OIDC token with repository and branch claims
|
v
AWS STS assumes lead-de-dev-jobs-artifact-publisher for a short session
|
v
s3://lead-de-dev-artifacts/glue-artifacts/<commit SHA>/
- brewery_pipeline-<version>-py3-none-any.whl
- glue_job.py
|
v
Terraform receives the selected commit SHA as a reviewed input
|
v
AWS Glue job uses that script and that wheel exactly
```

Why OIDC replaces AWS keys

Stored AWS access keys are long-lived secrets. They can be accidentally leaked, copied into a new workflow, or remain usable long after a developer intended. OIDC changes this exchange. GitHub produces a signed identity token for the individual workflow run; AWS validates the issuer, audience, repository, and branch in the role trust policy; then AWS STS returns a temporary session. The session expires automatically.

The role policy is a second check. Even a valid main-branch session can only list the glue-artifacts prefix and put objects below that prefix. Authentication answers who may act. Authorization answers what that identity may do.

The two independent controls

Control	Where	Question answered	Current value
Trust policy	github_oidc.tf	Who may assume the role?	sk03 167/data-platform-jobs on main
Identity policy	github_oidc.tf	What can the role do?	list and write glue-artifacts only
Workflow condition	test.yml	When do we request AWS credentials?	direct push to main only
Artifact path	test.yml	Which build is being published?	the triggering GITHUB_SHA

3. Repository responsibilities and branch model

Infrastructure repository

/Users/shivanshk/Documents/pdev/learn-terraform-get-started-aws owns AWS account resources. Its modules create the data lake buckets and the artifact bucket. Its root configuration creates operational resources, networking, and the GitHub OIDC publisher identity. Its workflow maps infrastructure branches to Terraform workspaces and tfvars files.

- It creates lead-de-dev-artifacts and its staged/prod equivalents.
- It owns the policy that makes the code publisher deliberately narrow.
- It will own the future aws_glue_job resource and the distinct Glue runtime role.
- It uses Terraform workspaces for state isolation and tfvars for explicit environment names.

Data jobs repository

/Users/shivanshk/Documents/pdev/data-platform-jobs owns executable behaviour. It does not create AWS resources. It packages reusable Python as a wheel, keeps the Glue entry point separate, and proves local behaviour with fixed tests. Its GitHub Actions workflow is the delivery pipeline for tested code.

- It fetches and transforms Open Brewery data from Bronze to Silver to Gold Delta tables.
- It builds a normal Python wheel from src/brewery_pipeline.
- It delivers glue/glue_job.py separately because that file is the Glue script entry point.
- It publishes only after a main push has passed lint, test, and packaging.

Environment belongs to infrastructure

A code commit is not itself a development, staging, or production deployment. The code repository currently releases its successful main build to the development artifact prefix. A later infrastructure PR decides which

immutable artifact SHA a dev, staging, or production Glue job should use. This keeps application source history simple and makes environment promotion an explicit infrastructure review.

Promotion example

```
Code repo main commit 4a1b...
-> publishes s3://lead-de-dev-artifacts/glue-artifacts/4a1b.../

Infrastructure dev change
-> sets glue_artifact_sha = "4a1b..."
-> Glue dev runs that artifact

After acceptance, staging infrastructure change
-> references the same reviewed source version from the staging artifact distribution strategy
-> Glue staging runs the promoted version

Production follows only after a reviewed production change.
```

The current project has only a dev publisher and a dev artifact bucket flow. Before staging or production publishing is added, decide whether promotion copies verified artifacts between buckets, lets a separate restricted publisher publish to each environment, or uses a central immutable release bucket. Do not silently broaden the dev publisher role.

4. Secure artifact publishing implementation

Infrastructure side: `github_oidc.tf`

The OIDC provider is account-wide, so the Terraform resources use `count = var.environment == "dev" ? 1 : 0`. This ensures a staging or production Terraform apply does not attempt to create a second instance of the same GitHub identity provider. Because `count` turns an object into a collection, references use `[0]` on the one instance created in `dev`.

The assume-role policy has two conditions. The audience must be `sts.amazonaws.com`, preventing the token from being used for a different audience. The subject must exactly name the repository and main branch. A workflow from another branch, another fork, or another repository does not satisfy the condition.

The publishing policy differentiates between a bucket-level permission and an object-level permission. `ListBucket` must use the bucket ARN and is narrowed by `s3:prefix`. `PutObject`, `AbortMultipartUpload`, and `ListMultipartUploadParts` use object ARNs below `glue-artifacts/*`. The role has no `GetObject`, `DeleteObject`, or `write` permission elsewhere.

Code side: `.github/workflows/test.yml`

The workflow always runs the quality gate. The two AWS steps have their own if condition; therefore GitHub still executes tests for a pull request while it skips AWS authentication and S3 upload. This is visible as skipped steps in the pull-request Action run, which is correct behaviour, not a failed publish.

`aws-actions/configure-aws-credentials` exchanges the GitHub OIDC token for temporary AWS credentials. `allowed-account-ids` catches an accidental role assumption from the wrong account. `role-session-name` gives CloudTrail and AWS audit views a run-specific label.

The upload step uses `GITHUB_SHA`, not a branch name, a build number, or `latest`. The SHA makes the S3 folder stable and reviewable. The preflight list check rejects an accidental attempt to publish again to an already occupied commit path.

Artifact layout

```
s3://lead-de-dev-artifacts/  
glue-artifacts/  
4a1b2c3d.../  
  brewery_pipeline-0.1.0-py3-none-any.whl  
  glue_job.py
```

Why the build uses two files

File	Produced from	Loaded by Glue as	Reason
glue_job.py	glue/glue_job.py	the job command script	keeps Glue-specific startup small and explicit
brewery_pipeline wheel	src/brewery_pipeline	an additional Python package	keeps transformation logic importable and testable outside Glue

5. What the current implementation guarantees and does not guarantee

Guaranteed by the current configuration

- Only a token with the configured repository, branch, and audience claims can assume the publisher role.
- Only the development environment creates the account-wide OIDC provider and this dev publisher role.
- The publisher can list only the glue-artifacts prefix and write only objects under that prefix.
- A pull request cannot run the OIDC credentials or publish steps because the event is not a push to main.
- The workflow exits before upload if it sees any object already in the SHA-specific prefix.
- S3 bucket versioning is enabled, preserving old versions if an object is overwritten by a principal that has permission to do so.

Important limitation: versioned does not mean WORM immutable

The current workflow enforces immutability as a delivery convention: it uses an unchanging commit path and refuses to overwrite a non-empty path. S3 versioning retains prior versions, but by itself does not prevent an identity with PutObject from creating a new current version at the same key. Do not claim a compliance-grade write-once guarantee from this design alone.

For a learning platform, the SHA path and refusal check are a strong practical milestone. For a production-grade release system, decide whether you need a release manifest with content hashes, S3 Object Lock retention, a separate release bucket and policy, or an artifact registry. Any choice must account for recovery from a partial upload and should be tested before being called immutable.

Important limitation: Terraform CI still uses stored AWS keys

The data-jobs publisher has no stored AWS keys. The infrastructure workflow still reads `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` from GitHub Secrets. This is a separate identity boundary. It is not safe to replace those infrastructure credentials with the narrow artifact publisher role because Terraform needs a broad set of permissions to manage the platform.

A later improvement can give the infrastructure repository its own OIDC deployment role. Bootstrap must be planned carefully: use the existing credential path to create and validate the new deployment role, then migrate the Terraform workflow, then remove the old GitHub secrets after a successful OIDC apply. That migration is intentionally not part of the artifact publisher role.

Important limitation: a library dependency must be reproducible

The package declares `requests==2.32.3`. AWS Glue 5.1 currently includes a `requests` version, but a runtime's preinstalled package list is a service detail and should not be your release contract. For a durable deployment, bundle and pin all non-Glue dependencies as a Glue-compatible wheel artifact or virtual environment. AWS recommends frozen artifacts such as a zip of wheels rather than resolving packages from PyPI during every job run.

6. Next platform milestone: a Glue job pinned to an artifact

What changes when a Glue job is added

Publishing puts tested files in a secure location. It does not cause a job to execute. Terraform must create a job definition that names the exact script and library location, chooses a Glue runtime, provides expected job arguments, and supplies an execution role. The execution role is deliberately different from the publisher role.

Recommended runtime alignment

Use AWS Glue 5.1 for the first deployed job after confirming it is available in the target region and provider version. Glue 5.1 uses Python 3.11, Spark 3.5.6, and includes Delta Lake 3.3.2. The current local project declares Delta 3.3.2, which aligns. Its development PySpark dependency is 3.5.7; align the local test dependency to the Glue runtime's exact supported Spark version before relying on a local test as runtime compatibility evidence.

The code uses DeltaTable APIs, so set `--datalake-formats` to `delta` and provide the Delta Spark configuration. The Glue script should receive the pipeline wheel with `--extra-py-files` or, preferably for explicit package installation, through the dependency mechanism chosen in the dependency packaging decision.

Artifact pinning contract

Input	Why Terraform needs it	Example
<code>glue_artifact_sha</code>	the immutable source revision selected for execution	<code>4a1b2c3d...</code>
<code>glue_artifact_wheel_filename</code>	the exact built distribution file under that revision	<code>brewery_pipeline-0.1.0-py3-none-any.whl</code>
<code>environment</code>	selects data locations and bucket names	<code>dev</code>
<code>batch_id</code> at start	identifies one logical run and makes retries safe	<code>brewery-page-1-2026-09-30</code>

Do not ask Terraform to infer a SHA by reading S3 latest. The SHA must be a reviewed input in the infrastructure change. This is the decision that turns a built artifact into deployed code.

Execution role requirements

- Trust AWS Glue service, not GitHub Actions.
- Read exactly the selected script and wheel in the artifact prefix.
- Read, write, and delete files necessary for Delta operations in the dev Bronze, Silver, and Gold buckets.
- Write required Glue temporary data and CloudWatch job logs.
- Have no publishing permission and no access to staging or production data during the dev rollout.

What the Glue job does at runtime

1. Glue starts the script selected by Terraform and adds the selected package to Python's import path.

2. glue_job.py reads JOB_NAME, environment, page, per_page, and batch_id with getResolvedOptions.
3. The script builds a GlueContext and Spark session, runs the pipeline, derives environment-specific S3 table paths, writes Bronze and quality outputs, updates Silver, refreshes Gold, and commits the Glue job.
4. The Delta helpers turn a retry using the same source ID and batch ID into an insert-only no-op for Bronze and audit tables, while Silver is refreshed by brewery ID.

7. Runbook from the current state

Phase 1: reconcile the corrected main branch design

The working trees recorded for this guide already show the corrected refs/heads/main values. If the currently deployed Terraform role still trusts refs/heads/dev, the code publish must wait until this infrastructure correction is applied.

1. In the infrastructure repository, confirm github_oidc.tf names repo:sk03 167/data-platform-jobs:ref:refs/heads/main.
2. Run terraform fmt -check -recursive. Open a pull request targeting the infrastructure dev branch and review the Terraform plan.
3. Merge into infrastructure dev. Verify the Terraform Automated Apply updates the IAM trust policy successfully.
4. In the code repository, confirm .github/workflows/test.yml triggers on main and both AWS steps test refs/heads/main.
5. Merge the code workflow change into the code repository main branch.

Phase 2: verify the first publish

1. Open the Action triggered by the main push, not the earlier pull-request Action run.
2. Confirm Configure AWS credentials for artifact publishing is green. A failure here indicates a trust-policy claim mismatch, incorrect role ARN, disabled OIDC permission, or wrong AWS account.
3. Confirm Publish immutable Glue artifacts is green and captures the printed S3 list.
4. Record the triggering full commit SHA and exact wheel filename from the logs.
5. Use the AWS S3 console or aws s3 ls to verify the exact prefix contains glue_job.py and one wheel.

Phase 3: prepare the Glue deployment change

1. Choose the exact published SHA, not a branch name or a moving alias.
2. Add the Glue execution role, its constrained data and artifact policy, and aws_glue_job to the infrastructure repository. A reference implementation appears in Appendix C.
3. Add the artifact SHA and wheel filename to the dev tfvars file only after they exist in S3.
4. Plan against the dev workspace. Review both the IAM policy scope and each S3 path in command.script_location and --extra-py-files.
5. Apply through the infrastructure dev merge flow, then run the job manually with a unique batch_id.

Phase 4: prove the deployed job

1. Run the Glue job with a small page and per_page value.
2. Inspect CloudWatch logs for import, API, Spark, Delta, and IAM errors.
3. Verify the expected Bronze, quarantine, null report, duplicate ID, curated, and Gold Delta paths in the dev buckets.
4. Run the exact same job arguments a second time. Verify the retry-safe table does not duplicate Bronze rows.
5. Only then promote the design to a distinct staging/prod artifact and execution pattern.

8. Troubleshooting and verification

Pull request shows AWS steps as skipped

This is expected. The current workflow deliberately requires a push event and refs/heads/main. Pull requests still prove quality but do not request AWS credentials. The fix is not to loosen the if condition; merge the approved pull request and inspect the new main push run.

Merge into main did not trigger a publish run

Check the workflow file on the main branch and verify that the merge commit includes the workflow change. Confirm the event is a push to main. If the workflow used branches: dev or tests refs/heads/dev, that configuration represents the older incorrect branch assumption and will not run for this repository. Replace both trigger and conditions with main, then merge that correction.

Configure AWS credentials fails

Likely cause	Evidence	Resolution
Trust subject names the wrong branch	AccessDenied from AssumeRoleWithWebIdentity	Make trust policy and workflow both use refs/heads/main, then apply the Terraform change first.
id-token permission missing	Action says it cannot request an ID token	Set permissions.id-token: write at workflow level or job level.
Role ARN or account is wrong	allowed-account-ids failure or missing role	Use Terraform's dev_jobs_artifact_publisher_role_arn output exactly.
Provider was not applied	role absent in IAM	Review the infrastructure dev apply before retesting the jobs workflow.

Publish rejects an existing path

This is the intended protection against overwrite. First inspect that prefix. If both files are correct, treat the prior publish as the artifact of record and use its SHA. If it is partial or incorrect, do not silently overwrite it. Investigate the workflow failure, fix the source, and publish a new commit SHA. This preserves an auditable lineage.

Glue cannot import `brewery_pipeline`

Check that Terraform uses the exact wheel filename shown by the publish workflow. Verify Glue sees the library location in its job arguments. Then confirm the wheel contains `brewery_pipeline` at its root. If the import succeeds locally but not in Glue, examine the packaging mode and any unresolved third-party dependency such as requests. Prefer a frozen dependency bundle or approved Glue virtual environment for a stable production-like run.

Glue Delta errors

Verify the Glue version and Delta configuration together. The job needs the Delta data lake framework and the associated Spark configuration. Also verify the execution role can list and write all required objects in the data-bucket paths; Delta writes metadata and data files and may remove superseded files during a merge or overwrite.

9. Security hardening backlog

Priority	Improvement	Why it matters	When
P0	Apply the main-branch trust correction before code publishing	prevents a predictable OIDC AssumeRole failure	now
P0	Create distinct Glue execution role	publisher must never gain data-plane permissions	before Glue job
P1	Bundle runtime dependencies as frozen wheels or a Glue venv	prevents runtime resolution and version drift	before production-like runs
P1	Migrate Terraform CI to a separate OIDC deployment role	removes remaining stored AWS credentials from GitHub	after safe bootstrap plan
P1	Pin GitHub Actions to reviewed commit SHAs	tags can move; a commit SHA fixes action code	security hardening pass
P2	Add a release manifest containing SHA256 digests	makes artifact integrity independently verifiable	after first publish
P2	Evaluate WORM / retention requirements	S3 versioning alone is not tamper-proof immutability	before compliance claim
P2	Add staging/prod publisher and promotion design	prevents accidental dev-role expansion	after dev Glue works
P2	Add data quality and job failure alarms	turns a working job into an operable service	after first successful job

Review questions before each promotion

- Which exact commit SHA is being deployed, and where was it tested?
- Which role is taking the action, and is it the smallest correct role?
- Which bucket and prefix does the role touch?
- Does the change introduce a mutable latest pointer or an unpinned dependency?
- Can a retry be identified and made safe with the same batch ID?
- What would prove success, and where would an operator look first if it failed?

Appendix A. Infrastructure repository file reference

This appendix records the current working-tree contents of the infrastructure repository. Each file is followed by an explanation of its purpose. The book treats these as a review snapshot; the future Glue reference appears separately in Appendix C rather than pretending it is already deployed.

providers.tf

Repository: infrastructure. **Purpose:** Terraform and provider bootstrap

Declares the Terraform version and AWS provider range, configures the remote S3 state backend, retains DynamoDB locking for the existing state model, and sets the AWS region. The backend must exist before normal Terraform deployment can work.

Current working-tree content:

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

backend "s3" {
  bucket     = "lead-de-global-tfstate-bucket" # Must be globally unique, change 'yourname'
  key        = "data-platform/terraform.tfstate"
  region     = "us-east-1"
  encrypt    = true
  dynamodb_table = "terraform-state-lock" # Retained for existing workspace locking.
}

}

provider "aws" {
  region = "us-east-1"
}
```

variables.tf

Repository: infrastructure. Purpose: Root input contract

Defines environment selection plus optional secret, notification, and alarm inputs. The environment validation protects resource naming from unrecognized strings.

Current working-tree content:

```
variable "db_password" {
  type      = string
  sensitive = true
  nullable  = true
  default   = null
  description = "Optional lab secret. Set through TF_VAR_db_password, never a committed tfvars file."
}

variable "alert_email_addresses" {
  type      = set(string)
  default   = []
  description = "Optional confirmed email recipients for data-operations alerts."
}

variable "gold_minimum_object_count" {
  type      = number
  default   = 1
  description = "Alert when daily Gold object count drops below this value."
}

variable "environment" {
  type      = string
  description = "Deployment target: dev, staging, or prod."

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging, or prod."
  }
}
```

main.tf

Repository: infrastructure. Purpose: Root platform composition

Instantiates the data lake and artifact bucket modules, then defines the database secret, operations SNS topic, subscriptions, and the Gold bucket object-count alarm.

Current working-tree content:

```
# 1. Instantiate the modular architecture
module "data_lake" {
  source      = "../modules/data_lake"
  environment = var.environment
}
module "glue_artifacts" {
  source      = "../modules/artifact_bucket"
  environment = var.environment
}

# 2. Secret Isolation: Securely vault operational passwords
resource "aws_secretsmanager_secret" "db_secret" {
  name                = "${var.environment}-lakehouse-db-credentials" # DYNAMIC NAME
  recovery_window_in_days = var.environment == "dev" ? 0 : 7
}

resource "aws_secretsmanager_secret_version" "db_secret_val" {
  count = var.db_password == null ? 0 : 1
  secret_id = aws_secretsmanager_secret.db_secret.id
  secret_string = jsonencode({
    username = "lakehouse_admin"
    password = var.db_password
  })
}

# 3. Operational Monitoring: SNS Alert Topic for Data Ops
resource "aws_sns_topic" "data_ops_alerts" {
  name = "${var.environment}-data-ops-lakehouse-alerts"
}

resource "aws_sns_topic_subscription" "email_alerts" {
  for_each = var.alert_email_addresses
  topic_arn = aws_sns_topic.data_ops_alerts.arn
  protocol = "email"
  endpoint = each.value
}

# 4. Proactive Alerting: CloudWatch Alarm tracking production gold tier integrity
resource "aws_cloudwatch_metric_alarm" "gold_data_loss_alarm" {
  alarm_name        = "${var.environment}-gold-bucket-integrity-alert"
  comparison_operator = "LessThanThreshold"
  evaluation_periods = 1
  metric_name       = "NumberOfObjects"
  namespace         = "AWS/S3"
  period            = 86400 # S3 storage metrics are emitted daily.
  statistic         = "Average"
  threshold         = var.gold_minimum_object_count
  alarm_description = "Fires instantly if files inside the production Gold layer are dropped or deleted."
  alarm_actions     = [aws_sns_topic.data_ops_alerts.arn]
  treat_missing_data = "breaching"

  dimensions = {
    BucketName = "lead-de-${var.environment}-gold-data"
    StorageType = "AllStorageTypes"
  }
}
```

networking.tf

Repository: infrastructure. Purpose: Learning VPC and S3 private route

Creates a VPC, one private subnet, its route table, and an S3 gateway endpoint. It is not yet connected to a Glue job; later Glue VPC execution needs additional networking and security-group decisions.

Current working-tree content:

```
# networking.tf

# 1. Base VPC Layout
resource "aws_vpc" "data_platform_vpc" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support = true
  tags            = { Name = "${var.environment}-data-platform-vpc" }
}

# 2. Private Subnet for Compute Clusters (Databricks/Airflow)
resource "aws_subnet" "private_compute_a" {
  vpc_id            = aws_vpc.data_platform_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "us-east-1a"
  tags              = { Name = "${var.environment}-private-compute-1a" }
}

# 3. Route Table for Private Subnets
resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.data_platform_vpc.id
  tags   = { Name = "${var.environment}-private-rt" }
}

resource "aws_route_table_association" "private_assoc" {
  subnet_id = aws_subnet.private_compute_a.id
  route_table_id = aws_route_table.private_rt.id
}

# 4. S3 VPC Gateway Endpoint (Routes traffic internally to your S3 storage tiers)
resource "aws_vpc_endpoint" "s3_endpoint" {
  vpc_id            = aws_vpc.data_platform_vpc.id
  service_name       = "com.amazonaws.us-east-1.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids    = [aws_route_table.private_rt.id]
  tags              = { Name = "${var.environment}-s3-vpc-endpoint" }
}
```

github_oidc.tf

Repository: infrastructure. Purpose: Dev GitHub artifact publisher identity

Creates the GitHub OIDC provider only in dev, trusts only data-platform-jobs/main, grants only the artifact-prefix list/write actions, and exposes the role ARN for the jobs workflow.

Current working-tree content:

```
resource "aws_iam_openid_connect_provider" "github_actions" {
  count      = var.environment == "dev" ? 1 : 0
  url       = "https://token.actions.githubusercontent.com"
  client_id_list = ["sts.amazonaws.com"]
}

data "aws_iam_policy_document" "github_jobs_publisher_assume_role" {
  count = var.environment == "dev" ? 1 : 0
  statement {
    effect = "Allow"
    actions = ["sts:AssumeRoleWithWebIdentity"]

    principals {
      type        = "Federated"
      identifiers = [aws_iam_openid_connect_provider.github_actions[0].arn]
    }

    condition {
      test     = "StringEquals"
      variable = "token.actions.githubusercontent.com:aud"
      values   = ["sts.amazonaws.com"]
    }

    condition {
      test     = "StringEquals"
      variable = "token.actions.githubusercontent.com:sub"
      values   = ["repo:sk03167/data-platform-jobs:ref:refs/heads/main"]
    }
  }
}

resource "aws_iam_role" "dev_jobs_artifact_publisher" {
  count      = var.environment == "dev" ? 1 : 0
  name       = "lead-de-dev-jobs-artifact-publisher"
  assume_role_policy = data.aws_iam_policy_document.github_jobs_publisher_assume_role[0].json
}

data "aws_iam_policy_document" "dev_jobs_artifact_publish" {
  count = var.environment == "dev" ? 1 : 0

  statement {
    sid = "ListGlueArtifactPrefix"
    effect = "Allow"
    actions = ["s3:ListBucket"]
    resources = [module.glue_artifacts.bucket_arn]

    condition {
      test     = "StringLike"
      variable = "s3:prefix"
      values   = ["glue-artifacts/**"]
    }
  }

  statement {
    sid = "WriteGlueArtifactsOnly"
    effect = "Allow"
    actions = [
      "s3:PutObject",
      "s3:AbortMultipartUpload",
      "s3:ListMultipartUploadParts",
    ]
    resources = ["${module.glue_artifacts.bucket_arn}/glue-artifacts/**"]
  }
}

resource "aws_iam_role_policy" "dev_jobs_artifact_publish" {
  count = var.environment == "dev" ? 1 : 0
  name = "write-glue-artifacts-only"
  role = aws_iam_role.dev_jobs_artifact_publisher[0].id
  policy = data.aws_iam_policy_document.dev_jobs_artifact_publish[0].json
}
```

```
output "dev_jobs_artifact_publisher_role_arn" {  
  description = "IAM role ARN assumed by data-platform-jobs CI to publish dev Glue artifacts."  
  value       = try(aws_iam_role.dev_jobs_artifact_publisher[0].arn, null)  
}
```

modules/artifact_bucket/main.tf

Repository: infrastructure. Purpose: Artifact bucket implementation

Creates one environment-specific artifact bucket with public access blocked, versioning, and SSE-S3 encryption. `force_destroy` is allowed only in dev.

Current working-tree content:

```
resource "aws_s3_bucket" "this" {
  bucket      = "lead-de-${var.environment}-artifacts"
  force_destroy = var.environment == "dev"

  tags = {
    Environment = var.environment
    Purpose     = "glue-artifacts"
    ManagedBy   = "Terraform"
  }
}

resource "aws_s3_bucket_public_access_block" "this" {
  bucket = aws_s3_bucket.this.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "this" {
  bucket = aws_s3_bucket.this.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "this" {
  bucket = aws_s3_bucket.this.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
```

modules/artifact_bucket/variables.tf

Repository: infrastructure. Purpose: Artifact module input

Accepts and validates the target environment for artifact-bucket naming.

Current working-tree content:

```
variable "environment" {  
  type    = string  
  description = "Deployment environment for this artifact bucket."  
  
  validation {  
    condition     = contains(["dev", "staging", "prod"], var.environment)  
    error_message = "environment must be dev, staging, or prod."  
  }  
}
```

modules/artifact_bucket/outputs.tf

Repository: infrastructure. Purpose: Artifact module contract

Exports the bucket name and ARN. github_oidc.tf uses the ARN instead of reconstructing it, so the policy remains tied to this module's resource.

Current working-tree content:

```
output "bucket_name" {
  description = "Name of the environment's Glue artifact bucket."
  value      = aws_s3_bucket.this.bucket
}

output "bucket_arn" {
  description = "ARN of the environment's Glue artifact bucket."
  value      = aws_s3_bucket.this.arn
}
```

modules/data_lake/main.tf

Repository: infrastructure. Purpose: Medallion bucket implementation

Uses `for_each` to create Bronze, Silver, and Gold buckets and applies public access blocking, metrics, versioning, encryption, and lifecycle rules.

Current working-tree content:

```
resource "aws_s3_bucket" "lake_bucket" {
  for_each = toset(var.lake_layers)

  bucket      = "lead-de-${var.environment}-${each.value}-data"
  force_destroy = var.environment == "dev"

  tags = {
    Environment = var.environment
    Layer       = each.value
    ManagedBy   = "Terraform"
  }
}

resource "aws_s3_bucket_public_access_block" "block_public" {
  for_each = toset(var.lake_layers)
  bucket   = aws_s3_bucket.lake_bucket[each.value].id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

# Request metrics are retained for future request-rate/error alarms.
resource "aws_s3_bucket_metric" "bucket_metrics" {
  for_each = toset(var.lake_layers)
  bucket   = aws_s3_bucket.lake_bucket[each.value].id
  name     = "EntireBucket"
}

resource "aws_s3_bucket_versioning" "lake_versioning" {
  for_each = toset(var.lake_layers)
  bucket   = aws_s3_bucket.lake_bucket[each.value].id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "lake_encryption" {
  for_each = toset(var.lake_layers)
  bucket   = aws_s3_bucket.lake_bucket[each.value].id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_lifecycle_configuration" "lake_lifecycle" {
  for_each = toset(var.lake_layers)
  bucket   = aws_s3_bucket.lake_bucket[each.value].id

  rule {
    id = "expire-noncurrent-versions"
    status = "Enabled"
    filter {}

    noncurrent_version_expiration {
      noncurrent_days = 30
    }
  }
}
```

modules/data_lake/variables.tf

Repository: infrastructure. Purpose: Data lake module inputs

Defines the environment and list of medallion layer names. The comment mentioning Platinum does not match the current default list, which remains Bronze, Silver, Gold.

Current working-tree content:

```
variable "environment" {
  type    = string
  description = "Explicit resource configuration target (dev, staging, prod)."
}

variable "lake_layers" {
  type    = list(string)
  default = ["bronze", "silver", "gold"] # ADDED PLATINUM HERE
  description = "Medallion data architecture tiers"
}
```

environments/dev.tfvars

Repository: infrastructure. Purpose: Development explicit inputs

Selects environment = dev. Future Glue artifact pin values belong here once the dev artifact exists.

Current working-tree content:

```
environment = "dev"
```

environments/staging.tfvars

Repository: infrastructure. Purpose: Staging explicit inputs

Selects environment = staging. It should not receive the dev-only publisher or Glue job until a separate promotion design is implemented.

Current working-tree content:

```
environment = "staging"
```

environments/prod.tfvars

Repository: infrastructure. Purpose: Production explicit inputs

Selects environment = prod. It is intentionally minimal at this learning stage.

Current working-tree content:

```
environment = "prod"
```

.github/workflows/terraform-ci.yml

Repository: infrastructure. Purpose: Infrastructure CI/CD

Plans pull requests and applies pushes across dev, staging, and main. It currently uses stored AWS secrets, which is intentionally tracked as separate future work.

Current working-tree content:

```
name: "Terraform Data Platform Multi-Env CI/CD"

on:
  push:
    branches:
      - dev
      - staging
      - main
  pull_request:
    branches:
      - dev
      - staging
      - main

permissions:
  contents: read

jobs:
  terraform:
    name: "Data Infrastructure Deployment Pipeline"
    runs-on: ubuntu-latest
    env:
      AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
      AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
      AWS_REGION: "us-east-1"
      TF_VAR_db_password: ${ secrets.DB_PASSWORD }

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.7.0

      - name: Terraform Format Check
        run: terraform fmt -check -recursive

      # Map the PR base branch or pushed branch to an explicit deployment environment.
      - name: Select target environment
        run: |
          if [ "${{ github.base_ref }}" = "main" ] || [ "${{ github.ref }}" = "refs/heads/main" ]; then
            echo "TARGET_ENV=prod" >> $GITHUB_ENV
          elif [ "${{ github.base_ref }}" = "staging" ] || [ "${{ github.ref }}" = "refs/heads/staging" ]; then
            echo "TARGET_ENV=staging" >> $GITHUB_ENV
          else
            echo "TARGET_ENV=dev" >> $GITHUB_ENV
          fi

      - name: Terraform Init
        run: terraform init -no-color

      # Workspaces partition the shared backend state. The tfvars file below
      # separately makes the resource configuration explicit.
      - name: Select Terraform workspace
        run: |
          terraform workspace select "$TARGET_ENV" || terraform workspace new "$TARGET_ENV"

      # Automated Quality Control: Run Plan previews on all Pull Requests
      - name: Terraform Plan Preview
        if: github.event_name == 'pull_request'
        run: terraform plan -no-color -input=false -var-file="environments/${TARGET_ENV}.tfvars"

      # ⚡ Production-Grade Isolation: Apply changes ONLY during direct pushes/merges to target tracks
      - name: Terraform Automated Apply
        if: github.event_name == 'push'
        run: terraform apply -auto-approve -no-color -input=false -var-file="environments/${TARGET_ENV}.tfvars"
```

.gitignore

Repository: infrastructure. Purpose: Infrastructure secret and state hygiene

Excludes provider data, local state, local terraform.tfvars, credential text files, and backup files from source control.

Current working-tree content:

```
.terraform/  
*.tfstate  
*.tfstate.backup  
terraform.tfvars  
creds.txt  
*.bak
```

README.md

Repository: infrastructure. Purpose: Infrastructure project handbook

Documents architecture decisions and current learning path. It needs a future documentation update to reflect the final main-to-dev artifact publisher decision.

Current working-tree content:

```
# Multi-Environment Medallion Data Platform Foundation
```

This repository is a learning-oriented AWS data-platform foundation. Terraform workspaces isolate state for `dev`, `staging`, and `prod`, while explicit environment files control resource names and configuration.

```
---
```

Project Repository Anatomy

```
```text
learn-terraform-get-started-aws/
├── .github/workflows/
│ └── terraform-ci.yml # CI/CD engine with Git branch-to-workspace auto-mapping
├── environments/ # Explicit dev, staging, and prod input values
├── modules/
│ ├── artifact_bucket/ # Versioned, encrypted Glue code-artifact storage
│ └── data_lake/
│ ├── main.tf # Loop-driven logic building dynamic storage tiers
│ └── variables.tf # Modular input configurations
├── .gitignore # Protects secrets from leaking to GitHub
├── main.tf # Secrets, alerts, and environment-specific resource configuration
├── providers.tf # Provider settings, Remote S3 Backend, and State Locking
├── variables.tf # Sensitive variable schemas
└── terraform.tfvars # Local secret values (Never Committed)
```
```

```
---
```

Code Deep Dive & Operational Rationale

1. `providers.tf` (Global Strategy & Remote State Backend)

Defines the shared S3 backend. Workspaces partition its state, while environment `tfvars` files explicitly set values used by resources.

```
```hcl
terraform {
 required_version = ">= 1.5.0"
 required_providers {
 aws = { source = "hashicorp/aws", version = "~> 5.0" }
 }

 backend "s3" {
 bucket = "lead-de-global-tfstate-bucket"
 key = "data-platform/terraform.tfstate"
 region = "us-east-1"
 encrypt = true
 dynamodb_table = "terraform-state-lock"
 }
}
```

```
provider "aws" { region = "us-east-1" }
```

#### ### 2. `modules/data\_lake/main.tf` (Modular Abstraction Engine)

Eliminates copy-pasted block redundancies. It iterates through the medallion layers and applies public-access blocking, versioning, encryption, and lifecycle protection. Only dev permits forced deletion.

```
```hcl
resource "aws_s3_bucket" "lake_bucket" {
  for_each = toset(var.lake_layers)
  bucket   = "lead-de-${var.environment}-${each.value}-data"
  force_destroy = var.environment == "dev"
  tags     = { Environment = var.environment, Layer = each.value, ManagedBy = "Terraform" }
}

resource "aws_s3_bucket_metric" "bucket_metrics" {
  for_each = toset(var.lake_layers)
  bucket   = aws_s3_bucket.lake_bucket[each.value].id
  name     = "EntireBucket"
}
```
```

#### ### 3. `main.tf` (Explicit Environment Configuration & Alerting)

Uses `var.environment`, supplied by an environment `tfvars` file, for resource naming. The selected Terraform workspace remains responsible only for state isolation.

```
```hcl
```

```

module "data_lake" {
  source      = "../modules/data_lake"
  environment = var.environment
}

resource "aws_secretsmanager_secret" "db_secret" {
  name            = "${var.environment}-lakehouse-db-credentials"
  recovery_window_in_days = var.environment == "dev" ? 0 : 7
}

resource "aws_secretsmanager_secret_version" "db_secret_val" {
  secret_id = aws_secretsmanager_secret.db_secret.id
  secret_string = jsonencode({ username = "lakehouse_admin", password = var.db_password })
}

resource "aws_sns_topic" "data_ops_alerts" { name = "${var.environment}-data-ops-alerts" }

resource "aws_cloudwatch_metric_alarm" "gold_data_loss_alarm" {
  alarm_name        = "${var.environment}-gold-bucket-integrity-alert"
  comparison_operator = "LessThanThreshold"
  evaluation_periods = "1"
  metric_name       = "NumberOfObjects"
  namespace         = "AWS/S3"
  period            = "86400"
  statistic         = "Average"
  threshold         = "1"
  alarm_actions     = [aws_sns_topic.data_ops_alerts.arn]
  dimensions       = { BucketName = "lead-de-${var.environment}-gold-data", StorageType = "AllStorageTypes" }
}

```

4. `.github/workflows/terraform-ci.yml` (GitOps Multi-Branch CI/CD Pipeline)
 Automates platform deployment governance using shell-mapping criteria to identify active base branches (`'dev'`, `'staging'`, `'main'`), match them immediately to the correct `"*Terraform Workspace"`, and isolate the infrastructure blast radius:

```

'''yaml
# Map the active branch to a workspace and matching explicit input file.
- name: Select Active Workspace
  run: |
    if [ "${{ github.base_ref }}" = "main" ] || [ "${{ github.ref }}" = "refs/heads/main" ]; then
      echo "TARGET_ENV=prod" >> $GITHUB_ENV
    elif [ "${{ github.base_ref }}" = "staging" ] || [ "${{ github.ref }}" = "refs/heads/staging" ]; then
      echo "TARGET_ENV=staging" >> $GITHUB_ENV
    else
      echo "TARGET_ENV=dev" >> $GITHUB_ENV
    fi

- name: Switch Workspace
  run: terraform workspace select ${ env:TARGET_ENV } || terraform workspace new ${ env:TARGET_ENV }

- name: Terraform Plan Preview
  run: terraform plan -var-file="environments/${ env:TARGET_ENV }.tfvars"
'''

```

5. `modules/artifact_bucket` (Deployable Code Storage)
 The data-jobs repository builds two deployable pieces: a Python wheel containing reusable pipeline code and a small Glue entry script. This module creates a separate bucket for those artifacts instead of mixing them with Bronze/Silver/Gold data:

```

'''text
lead-de-dev-artifacts
lead-de-staging-artifacts
lead-de-prod-artifacts
'''

```

The bucket blocks public access, enables versioning, uses S3-managed encryption, and permits `'force_destroy'` only in `'dev'`. Its name and ARN are module outputs so a later GitHub OIDC publishing role can receive access to this one bucket rather than broad S3 permissions.

Infrastructure Learning Path and Decisions

From implicit workspaces to explicit environment inputs
 The first version used `'terraform.workspace'` both to isolate state and to name resources. That worked for a small lab, but it made resource configuration depend on hidden CLI state: the same command could describe a different environment if the wrong workspace was selected.

The current design separates the concerns:

```

'''text
Terraform workspace → isolates remote state
environment tfvars  → explicitly selects resource configuration

```

...

Each environment file sets `environment = "dev", "staging", or "prod"`. Resources use `var.environment`; workspaces remain in the workflow to keep their state files separate. This makes a plan reviewable from its command line:

```
```bash
terraform workspace select dev
terraform plan -var-file=environments/dev.tfvars
```
```

Branches are deployment tracks, not merely labels
Terraform CI maps a PR base branch or a direct push to one matching workspace and `.tfvars` file:

```
```text
feature PR → dev → dev workspace + environments/dev.tfvars
dev → staging → staging workspace + environments/staging.tfvars
staging → main → prod workspace + environments/prod.tfvars
```
```

Pull requests run a plan. A merge/direct push to the target track runs the apply. This is why the Glue artifact bucket was planned on a feature branch, merged to `dev`, and created by the `dev` deployment pipeline rather than by a manual apply.

A source-control lesson: deployed is not the same as merged
While adding the artifact bucket, a plan against `feat/add\_vpc` showed `No changes`: its explicit-environment and VPC configuration had already been applied to `dev`. The same configuration was missing from `main`, however.

The correct fix was not to deploy the older `main` configuration again. The branch was first promoted into `main`, then the artifact-bucket feature was rebased onto that baseline. This preserved a one-to-one relationship between the deployed state, the approved branch history, and future plans.

Artifact delivery boundary
The data-jobs repository now lint-tests-builds the Glue script and wheel in CI, then stores a short-lived GitHub Actions artifact for inspection. This Terraform repository now provides the dev artifact bucket. The next step is restricted GitHub ODC publishing from `main` into immutable Git-SHA paths, followed by a Glue job that pins an exact published artifact.

Known backend follow-up
Terraform currently warns that the S3 backend `dynamodb\_table` parameter is deprecated. State locking still works today; a future maintenance change should migrate the backend to `use\_lockfile` after confirming the team no longer needs the DynamoDB lock table.

✂ The Enterprise Production Lifecycle Playbook

1. Initial State Backend Bootstrapping

Execute these administrative actions via the AWS CLI to provision the remote backend storage components before launching your code modules:

```
```bash
aws s3api create-bucket --bucket lead-de-global-tfstate-bucket --region us-east-1
aws s3api put-bucket-versioning --bucket lead-de-global-tfstate-bucket --versioning-configuration Status=Enabled
```
```

2. Multi-Branch Progression Model

To deploy updates securely without configuration drift, code modifications must flow sequentially up the environment branch ladder:

1. Developer cuts a local branch from `dev` ('feat/change-parameters').
2. Open a Pull Request targeting `dev` → CI selects the `Dev Workspace` and passes `environments/dev.tfvars` to `terraform plan`.
3. PR Merge to `dev` triggers `terraform apply` → updates roll out exclusively onto your Dev cloud assets.
4. Promote verified code from `dev` → `staging` branch via a new PR, which runs checks against the isolated `Staging Workspace` partition (`env:/staging/...`).
5. Final promotion from `staging` → `main` releases the fully validated updates cleanly onto the production architecture track.

3. Local Infrastructure Drift Remediation

If an asset is deleted or modified manually outside your pipeline controls via web dashboards, the decentralized state structure instantly captures and patches the drift:

```
```bash
1. Simulate an external deletion event
aws s3 rb s3://lead-de-dev-bronze-data --force

2. Audit reality against the code blueprint
terraform workspace select dev
terraform plan -var-file=environments/dev.tfvars

3. Heal the environment instantly
terraform apply -var-file=environments/dev.tfvars
```
```

4. Complete Environment Teardown

To practice sound resource cost optimization controls and prevent running up unexpected cloud utility fees when testing architectures, completely erase temporary environments using the teardown sequence:

```
```bash
terraform workspace select dev && terraform destroy -var-file=environments/dev.tfvars
terraform workspace select staging && terraform destroy -var-file=environments/staging.tfvars
terraform workspace select prod && terraform destroy -var-file=environments/prod.tfvars
terraform workspace select default
```
```

Future Platform Roadmap & Next Targets

As we scale this foundational architecture into a production-ready enterprise lakehouse, the following milestones represent our next targets:

Target 1: Networking & IAM Security Hardening

Objective: Remove the storage endpoints from the public internet entirely.

Execution: Wrap the medallion layers inside an **AWS VPC with Private Subnets**. Set up **S3 VPC Gateway Endpoints** so your compute clusters route data internally without traveling over public transit. Enforce strict **IAM Role Assume Policies** and Service Control Policies (SCPs) to establish ironclad Least Privilege limits.

* Target 2: Databricks & Snowflake Native Provider Integration

Objective: Provision actual computing structures, role assignments, and schemas cleanly through code.

Execution: Add the HashiCorp **Databricks and Snowflake Terraform Providers** to our global `providers.tf` configuration blocks. This will allow us to manage cluster policies, Unity Catalog metastores, virtual compute warehouses, access controls (RBAC), and table schemas natively alongside our cloud storage foundation using standard GitOps pipelines.

Target 3: Orchestration Infrastructure with IaC

Objective: Automate production data workflow pipelines using managed orchestrators.

Execution: Provision an **AWS MWAA (Managed Workflows for Apache Airflow)** instance or a **Prefect Cloud Agent** framework completely via Terraform.

This layer will manage the dynamic scheduling, execution dependencies, and pipeline workflows driving data moving across the Gold, Silver, and Bronze storage layers.

Appendix B. Data jobs repository file reference

This appendix records the current working-tree contents of the data jobs repository. The source package stays independent of AWS resource creation so that it remains locally testable. The workflow is the only code-repository component that needs AWS awareness, and it has only the role ARN, region, bucket name, and prefix convention necessary for publishing.

README.md

Repository: data-platform-jobs. Purpose: Repository introduction and operating contract

Explains the job's Bronze to Silver to Gold data flow, its packaging boundary, local development, test contract, and CI delivery model. Its current narrative should be updated after the first successful OIDC publish so it describes an implemented capability rather than a future step.

Current working-tree content:

```
# Data Platform Jobs
```

```
Deployable Python jobs for an AWS Glue learning data platform. This repository owns the data-product code, local tests, package build, and code CI. Its paired Terraform repository owns AWS infrastructure, IAM, and Glue job configuration.
```

```
## What the current job does
```

```
`brewery_pipeline` fetches a bounded page from the public Open Brewery DB API and produces a small Bronze → Silver → Gold Delta Lake pipeline:
```

```
```text
Open Brewery DB
 ↓
Bronze: typed raw records + ingestion metadata
 |
 └─ invalid required fields → quarantine
 ↓
Silver: valid, deduplicated brewery records
 ↓
Gold: country/type brewery summary and coordinate coverage
```
```

```
The Glue entry point accepts an `environment`, `page`, `per_page`, and stable `batch_id`. It resolves environment-specific S3 paths such as `lead-de-dev-bronze-data` and writes Delta tables there.
```

```
## Repository layout
```

```
```text
glue/glue_job.py AWS Glue entry script
src/brewery_pipeline/pipeline.py Reusable Bronze/Silver/Gold pipeline logic
tests/ Pytest suite and fixed input fixtures
pyproject.toml Package metadata and dependency definitions
.github/workflows/test.yml Lint, test, package-build, and artifact CI
```
```

```
The Glue script is intentionally outside `src/`. The wheel contains only reusable package code; Glue receives `glue_job.py` as its script and the wheel as an additional Python module.
```

```
## Local development
```

```
Prerequisites: Python 3.11 and Java 17. Spark needs the driver and worker to use the same Python minor version; the shared pytest fixture explicitly points both to the repository virtual environment.
```

```
```bash
python3.11 -m venv .venv
.venv/bin/python -m pip install -e ".[dev]"

JAVA_HOME=$(/usr/libexec/java_home -v 17) \
PATH="$JAVA_HOME/bin:$PATH" \
.venv/bin/python -m pytest -q
```
```

```
Run code-quality checks and build deployable artifacts locally with:
```

```
'''bash
.venv/bin/python -m ruff check .
.venv/bin/python -m build
'''
```

`dist/` contains a source distribution and the deployable wheel. It is ignored by Git because CI creates the authoritative artifact for each run.

Tests and data contract

Tests use small fixed JSON fixtures rather than the live API, so results are repeatable and do not depend on network availability or changing source data.

The current suite verifies:

- environment names resolve to the correct Bronze, Silver, and Gold S3 paths;
- valid API data becomes typed Bronze data with `batch\_id`, `ingested\_at`, and `ingested\_date` metadata;
- records missing required `id`, `name`, `city`, or `country` fields are quarantined with a clear reason;
- replaying the same source record with the same `batch\_id` does not duplicate the Bronze Delta table.

The `batch\_id` identifies one logical source extract. A retry must reuse the same value; for a manual run, `brewery-page-1-YYYY-MM-DD` is a clear format.

CI and artifact flow

GitHub Actions runs on pull requests and pushes to `main`:

```
'''text
Ruff lint → local Spark/Delta pytest suite → Python wheel build
→ temporary GitHub Actions artifact
'''
```

The workflow uploads `glue-artifacts`, containing the wheel and Glue script, for seven days. This validates packaging on a clean runner without AWS credentials. A later delivery step will publish immutable, Git-SHA-addressed artifacts to an S3 artifact bucket only after changes reach `main`.

What this project has taught me

- A production Glue job can have one executable script while still keeping testable pipeline code in a normal Python package.
- `pyproject.toml` is the source of truth for packaging and dependencies; editable installs are for development, while `python -m build` creates the actual wheel.
- Pytest fixtures provide controlled setup: JSON fixtures provide known source data, while the session-scoped Spark fixture supplies a reusable local Spark session.
- Delta `MERGE` logic and a caller-supplied batch identifier are the basis of retry-safe ingestion; a rerun is not automatically safe unless its identity is designed deliberately.
- CI must prove more than unit behavior: linting, a clean-environment Spark test, and a package build catch different classes of failure.
- The code and Terraform repositories have separate responsibilities: code CI creates tested artifacts, while infrastructure CI controls where and how those artifacts are deployed.

Current boundary

This repository does not create AWS resources or publish to S3 yet. The next phase provisions the artifact bucket and restricted GitHub OIDC publishing in Terraform, then pins a tested artifact version in the Glue job configuration.

pyproject.toml

Repository: data-platform-jobs. Purpose: Python build and dependency definition

Defines the wheel name/version, Python floor, runtime dependency requests, development test/build dependencies, src layout discovery, and pytest test path. The wheel filename used by Glue follows from its project name and version.

Current working-tree content:

```
[build-system]
requires = ["setuptools>=75"]
build-backend = "setuptools.build_meta"

[project]
name = "brewery-pipeline"
version = "0.1.0"
description = "Open Brewery batch data pipeline"
requires-python = ">=3.10"
dependencies = [
    "requests==2.32.3",
]

[project.optional-dependencies]
dev = [
    "delta-spark==3.3.2",
    "pyspark==3.5.7",
    "pytest==8.3.5",
    "ruff==0.11.0",
    "build==1.2.2.post1",
]

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

glue/glue_job.py

Repository: data-platform-jobs. Purpose: Glue script entry point

Creates the Glue Spark context, resolves command-line arguments, calls importable pipeline functions, writes results, refreshes Gold, emits a completion log, and commits the Glue job.

Current working-tree content:

```
"""AWS Glue entry point for the Open Brewery batch pipeline."""

import logging
import sys

from awsglue.context import GlueContext
from awsglue.job import Job
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext

from brewery_pipeline.pipeline import (
    build_paths,
    refresh_gold,
    run_pipeline,
    write_pipeline_outputs,
)

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")

def create_glue_spark():
    """Return the Spark session managed by this Glue job."""
    spark_context = SparkContext.getOrCreate()
    glue_context = GlueContext(spark_context)
    glue_context.spark_session.sparkContext.setLogLevel("WARN")
    return glue_context, glue_context.spark_session

def main() -> None:
    args = getResolvedOptions(
        sys.argv,
        ["JOB_NAME", "environment", "page", "per_page", "batch_id"],
    )

    glue_context, spark_session = create_glue_spark()
    job = Job(glue_context)
    job.init(args["JOB_NAME"], args)

    outputs = run_pipeline(
        spark_session,
        page=int(args["page"]),
        per_page=int(args["per_page"]),
        batch_id=args["batch_id"],
    )
    paths = build_paths(args["environment"])
    write_pipeline_outputs(spark_session, outputs, paths)
    refresh_gold(spark_session, paths["curated"], paths["gold"])

    logging.info("Completed batch %s", outputs["batch_id"])
    job.commit()

if __name__ == "__main__":
    main()
```

src/brewery_pipeline/__init__.py

Repository: data-platform-jobs. Purpose: Package marker

Makes brewery_pipeline an importable package. It is intentionally empty; its presence matters to package discovery and Python import semantics.

Current working-tree content:

src/brewery_pipeline/pipeline.py

Repository: data-platform-jobs. Purpose: Reusable pipeline implementation

Contains API retrieval, Bronze schema construction, quality handling, deduplication, curation, Gold aggregation, Delta write helpers, idempotent merge behaviour, and environment-derived S3 paths. The extensive inline learning comments are source documentation rather than runtime logic.

Current working-tree content:

```
"""AWS Glue version of the Open Brewery Bronze -> Silver -> Gold pipeline.
```

```
Design improvements made while extracting this from the notebook:
```

- The module is import-safe; execution happens only through main().
- A caller supplies the batch ID, making a logical retry traceable.
- Spark setup uses the session managed by AWS Glue.
- Delta write and Gold-refresh helpers receive their Spark session and paths explicitly instead of relying on notebook globals.

```
Bronze and audit outputs use insert-only Delta MERGES, so a retry with the same batch ID and source record ID becomes a no-op.
```

```
"""
```

```
import logging
import requests
from pyspark.sql.types import DateType, DoubleType, StringType, StructField, StructType, TimestampType
from delta.tables import DeltaTable
```

```
from pyspark.sql import functions as F
```

```
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
```

```
#Pipeline Helpers
```

```
def fetch_breweries(page: int, per_page: int) -> list[dict]:
    base_url = 'https://api.openbrewerydb.org/v1/breweries'
    headers = {"Content-Type": "application/json", "Accept": "application/json"}
    params = {"page": page, "per_page": per_page, }
```

```
try:
    response = requests.get(
        url=base_url, headers=headers, params=params, timeout=30
    )
    response.raise_for_status()
    return response.json()
except requests.exceptions.Timeout as err:
    logging.error(f"Request Timed Out Error: {err}")
    raise
except requests.exceptions.HTTPError as err:
    logging.error(f"Request HTTP Error : {err}")
    raise
except requests.exceptions.RequestException as err:
    logging.error(f"An error occurred: {err}")
    raise
```

```
# x=fetch_breweries(1,10)
# print(len(x),x[0]['id'])
```

```
def to_bronze_df(spark_session, records: list[dict], batch_id:str):
```

```
    """Apply bronze_schema and add ingested_at."""
```

```
    bronze_schema = StructType(
        [
            StructField('id',StringType(), True),
            StructField('name',StringType(),True),
            StructField("brewery_type", StringType(), True),
            StructField("address_1", StringType(), True),
            StructField("address_2", StringType(), True),
            StructField("address_3", StringType(), True),
            StructField("city", StringType(), True),
            StructField("state_province", StringType(), True),
            StructField("postal_code", StringType(), True),
            StructField("country", StringType(), True),
            StructField("longitude", DoubleType(), True),
            StructField("latitude", DoubleType(), True),
            StructField("phone", StringType(), True),
            StructField("website_url", StringType(), True),
            StructField("state", StringType(), True),
            StructField("street", StringType(), True),
            StructField("ingested_at", TimestampType(), True),
            StructField("ingested_date", DateType(), True),
```

```

        StructField("batch_id", StringType(), True),
    ]
)
bronze_df = spark_session.createDataFrame(records, schema=bronze_schema).withColumns(
    {
        "ingested_at": F.current_timestamp(),
        "ingested_date": F.current_date(),
        "batch_id": F.lit(batch_id)
    }
)
return bronze_df

# bronze_brews = to_bronze_df(x)
# bronze_brews.show()

def profile_nulls(bronze_df):
    """Return a one-row null-count report."""
    # [list comprehension](generator){single value is set comprehension}[key:value is dictionary comprehension]
    # for tuple comprehension, a generator is passed inside tuple(generator)
    # for row in x:
    #     for field in row.keys():
    #         yield field
    # below is just above loop written in a single line, yielded value first, then outer loop, then inner loop
    # all_fields = {field for row in x for field in row.keys()} #it is allowed to skip the generator's parenthesis here since it is the only input, kept for brevity's sake
    # print(all_fields)
    # Though I just realised above is moot because I need to use bronze_df only to create this

    null_cols_expression = [F.sum(F.col(input_col).isNull().cast('int')).alias(f'{input_col}_null_count') for input_col in bronze_df.columns]
    # print(null_cols_expression)
    return bronze_df.agg(F.first("batch_id").alias("batch_id"), F.count("").alias('total_row_count'), *null_cols_expression)

# profile_nulls(bronze_brews).show()

def split_valid_and_quarantine(bronze_df):
    """Return valid_df, quarantine_df"""
    from functools import reduce
    required_fields = ["id", "name", "city", "country"]
    missing_required = reduce(
        lambda left, right: left | right,
        [
            F.col(column).isNull() | (F.trim(F.col(column))=="")
            for column in required_fields
        ]
    )
    # bronze_df.filter(missing_required) this basically shows all rows where any of the required columns is null
    # so naturally if we take complement of it, it would show all rows where all of the required columns are not null
    # above statement happens because of Morgan's law of boolean algebra shown below
    # NOT (id missing OR name missing OR city missing OR country missing)
    # =
    # id present AND name present AND city present AND country present
    valid_df = bronze_df.filter(~missing_required)

    quarantine_df = bronze_df.filter(missing_required).withColumn('quarantine_reason', F.lit('a required column is missing'))

    return valid_df, quarantine_df

# print(valid_df.count())
# print(missing_required)
# a,b = split_valid_and_quarantine(bronze_brews)
# print(a.count(), b.count())

def deduplication(valid_df):
    duplicate_ids_df = valid_df.groupBy('id').agg(F.count("").alias('dupe_count'), F.first("batch_id").alias("batch_id")).filter(F.col('dupe_count')>1)
    # above it could also be "dupe_count > 1" but it cannot be "dupe_count">1 or lit(1), it needs to be one whole expression
    deduped_df = valid_df.dropDuplicates(['id'])
    return duplicate_ids_df, deduped_df
# c,d = deduplication(a)
# print(c.count(), d.count())

def build_curated(deduped_df):
    """Deduplicate IDs and apply curated transformations."""
    enriched_df = deduped_df.withColumns(
        {
            "name": F.trim(F.col("name")),
            "city": F.trim(F.col("city")),
            "country": F.trim(F.col("country")),
            "has_coordinates": (
                F.col("latitude").isNotNull()
                & F.col("longitude").isNotNull()
            ),
        }
    )
)

```

```

return enriched_df.select(
    "id",
    "name",
    "brewery_type",
    "city",
    "state_province",
    "postal_code",
    "country",
    "latitude",
    "longitude",
    "website_url",
    "ingested_at",
    "has_coordinates",
    "batch_id"
)

def build_gold(silver_df):
    aggregated = silver_df.groupBy("country", "brewery_type").agg(
        F.count("").alias("brewery_count"),
        F.sum(
            F.when(F.col("has_coordinates"), F.lit(1)).otherwise(F.lit(0))
        ).alias("breweries_with_coordinates"),
    )

    return (
        aggregated
        .withColumn(
            "coordinate_coverage_pct",
            F.col("breweries_with_coordinates")
            / F.col("brewery_count")
            * 100,
        )
        .orderBy(F.col("brewery_count").desc())
    )

def refresh_gold(spark_session, curated_path: str, gold_path: str) -> None:
    silver_df = spark_session.read.format("delta").load(curated_path)
    gold_df = build_gold(silver_df)
    overwrite_to_delta(gold_df, gold_path)
    # curated_df = build_curated(d)
    # print(curated_df.count())
    # curated_df.printSchema()

#Pipeline run definition
def run_pipeline(spark_session, page: int, per_page: int, batch_id: str) -> dict[str, object]:
    records = fetch_breweries(page, per_page)
    bronze_df = to_bronze_df(spark_session, records, batch_id)
    null_report_df = profile_nulls(bronze_df)
    valid_df, quarantine_df = split_valid_and_quarantine(bronze_df)
    duplicate_ids_df, deduped_df = deduplication(valid_df)
    curated_df = build_curated(deduped_df)
    # gold_df = build_gold(curated_df)
    return {
        "batch_id": batch_id,
        "bronze": bronze_df,
        "null_report": null_report_df,
        "valid": valid_df,
        "quarantine": quarantine_df,
        "duplicate_ids": duplicate_ids_df,
        "curated": curated_df,
        # "gold": gold_df
    }
    # Bronze is written before quality filtering.
    # Then profile, validate, curate, and write outputs.

#Pipeline Write helpers and main definition
def build_paths(environment: str) -> dict[str, str]:
    """Return the S3 Delta-table paths for one deployment environment."""
    return {
        "bronze": (
            f"s3://lead-de-{environment}-bronze-data/"
            "brewery_pipeline/bronze_breweries"
        ),
        "quarantine": (
            f"s3://lead-de-{environment}-bronze-data/"
            "brewery_pipeline/quarantined_breweries"
        ),
        "null_report": (
            f"s3://lead-de-{environment}-bronze-data/"
            "brewery_pipeline/null_reports"
        ),
        "duplicate_ids": (
            f"s3://lead-de-{environment}-bronze-data/"
            "brewery_pipeline/duplicate_ids"
        ),
    },

```

```

    "curated": (
        f"s3://lead-de-{environment}-silver-data/"
        "brewery_pipeline/curated_breweries"
    ),
    "gold": (
        f"s3://lead-de-{environment}-gold-data/"
        "brewery_pipeline/gold_brewery_summary"
    ),
}

def merge_curated(spark_session, df, path: str) -> None:
    """Insert new curated breweries and refresh existing ones by ID."""
    if DeltaTable.isDeltaTable(spark_session, path):
        (
            DeltaTable.forPath(spark_session, path)
            .alias("target")
            .merge(
                df.alias("source"),
                "target.id = source.id",
            )
            .whenMatchedUpdateAll()
            .whenNotMatchedInsertAll()
            .execute()
        )
    else:
        df.write.format("delta").mode("overwrite").save(path)

def merge_insert_only(
    spark_session,
    df,
    path: str,
    merge_condition: str,
    partition_columns: list[str] | None = None,
) -> None:
    """Insert only source rows not already recorded by the logical batch key."""
    if DeltaTable.isDeltaTable(spark_session, path):
        (
            DeltaTable.forPath(spark_session, path)
            .alias("target")
            .merge(df.alias("source"), merge_condition)
            .whenNotMatchedInsertAll()
            .execute()
        )
    return

writer = df.write.format("delta").mode("overwrite")
if partition_columns:
    writer = writer.partitionBy(*partition_columns)
writer.save(path)

def overwrite_to_delta(df, path: str) -> None:
    df.write.format("delta").mode("overwrite").option("overwriteSchema", True).save(path)

def write_pipeline_outputs(spark_session, outputs: dict, paths: dict[str, str]) -> None:
    """Persist one completed pipeline run to Delta tables."""

    # Retry-safe raw/audit writes. The Open Brewery source supplies a stable id.
    merge_insert_only(
        spark_session,
        outputs["bronze"],
        paths["bronze"],
        "target.batch_id = source.batch_id AND target.id = source.id",
        partition_columns=["ingested_date"],
    )
    merge_insert_only(
        spark_session,
        outputs["quarantine"],
        paths["quarantine"],
        "target.batch_id = source.batch_id AND target.id = source.id",
    )
    merge_insert_only(
        spark_session,
        outputs["null_report"],
        paths["null_report"],
        "target.batch_id = source.batch_id",
    )
    merge_insert_only(
        spark_session,
        outputs["duplicate_ids"],
        paths["duplicate_ids"],
        "target.batch_id = source.batch_id AND target.id = source.id",
    )
    # Merge curated breweries by id:
    # update existing IDs and insert new IDs.
    merge_curated(spark_session, outputs["curated"], paths["curated"])

```

tests/conftest.py

Repository: data-platform-jobs. Purpose: Shared Spark test fixture

Configures local Spark and Delta Lake once per pytest session. It forces driver and worker Python to the active test interpreter and stops Spark at the end.

Current working-tree content:

```
import pytest
from pyspark.sql import SparkSession
import os
import sys
from delta import configure_spark_with_delta_pip

@pytest.fixture(scope="session")
def spark():
    os.environ["PYSPARK_PYTHON"] = sys.executable
    os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
    builder = (
        SparkSession.builder
        .appName("brewery-pipeline-tests")
        .master("local[2]")
        .config("spark.sql.shuffle.partitions", "2")
        .config("spark.sql.session.timeZone", "UTC")
        .config(
            "spark.sql.extensions",
            "io.delta.sql.DeltaSparkSessionExtension",
        )
        .config(
            "spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog",
        )
    )

    session = configure_spark_with_delta_pip(builder).getOrCreate()

    yield session

    session.stop()
```

tests/test_bronze.py

Repository: data-platform-jobs. Purpose: Bronze and quarantine tests

Proves source fields and ingestion metadata are preserved, then proves a missing required ID is quarantined with the expected reason.

Current working-tree content:

```
import json
from pathlib import Path

from brewery_pipeline.pipeline import (
    split_valid_and_quarantine,
    to_bronze_df,
)

def test_to_bronze_df_preserves_source_data_and_adds_run_metadata(spark):
    fixture_path = Path(__file__).parent / "fixtures" / "breweries_valid.json"
    records = json.loads(fixture_path.read_text())

    bronze_df = to_bronze_df(
        spark,
        records,
        batch_id="test-batch-001",
    )

    row = bronze_df.first()

    assert bronze_df.count() == 1
    assert row["id"] == "test-brewery-001"
    assert row["name"] == "Test Brewery"
    assert row["batch_id"] == "test-batch-001"
    assert row["ingested_at"] is not None
    assert row["ingested_date"] is not None

def test_missing_required_id_is_quarantined(spark):
    fixture_path = Path(__file__).parent / "fixtures" / "breweries_invalid.json"
    records = json.loads(fixture_path.read_text())

    bronze_df = to_bronze_df(
        spark,
        records,
        batch_id="test-batch-invalid-001",
    )
    valid_df, quarantine_df = split_valid_and_quarantine(bronze_df)

    assert valid_df.count() == 0
    assert quarantine_df.count() == 1

    row = quarantine_df.first()
    assert row["id"] is None
    assert row["quarantine_reason"] == "a required column is missing"
```

tests/test_idempotency.py

Repository: data-platform-jobs. Purpose: Retry-safety test

Writes one batch twice into local Delta directories and verifies the Bronze target retains one row. This is the central proof for the insert-only batch/source key merge.

Current working-tree content:

```
import json
from pathlib import Path

from brewery_pipeline.pipeline import (
    build_curated,
    deduplication,
    profile_nulls,
    split_valid_and_quarantine,
    to_bronze_df,
    write_pipeline_outputs,
)

def test_replaying_same_batch_does_not_duplicate_bronze_rows(spark, tmp_path):
    fixture_path = Path(__file__).parent / "fixtures" / "breweries_valid.json"
    records = json.loads(fixture_path.read_text())

    bronze_df = to_bronze_df(
        spark,
        records,
        batch_id="test-batch-001",
    )
    valid_df, quarantine_df = split_valid_and_quarantine(bronze_df)
    duplicate_ids_df, deduped_df = deduplication(valid_df)

    outputs = {
        "bronze": bronze_df,
        "null_report": profile_nulls(bronze_df),
        "quarantine": quarantine_df,
        "duplicate_ids": duplicate_ids_df,
        "curated": build_curated(deduped_df),
    }
    paths = {
        "bronze": str(tmp_path / "bronze"),
        "quarantine": str(tmp_path / "quarantine"),
        "null_report": str(tmp_path / "null_report"),
        "duplicate_ids": str(tmp_path / "duplicate_ids"),
        "curated": str(tmp_path / "curated"),
    }

    write_pipeline_outputs(spark, outputs, paths)
    write_pipeline_outputs(spark, outputs, paths)

    written_bronze = spark.read.format("delta").load(paths["bronze"])

    assert written_bronze.count() == 1
    assert written_bronze.first()["batch_id"] == "test-batch-001"
```

tests/test_paths.py

Repository: data-platform-jobs. Purpose: Path contract test

Locks the mapping from an environment name to all required S3 Delta output paths. It catches accidental bucket/path naming changes early.

Current working-tree content:

```
from brewery_pipeline.pipeline import build_paths

def test_build_paths_uses_environment_specific_buckets():
    paths = build_paths("dev")

    assert paths == {
        "bronze": (
            "s3://lead-de-dev-bronze-data/"
            "brewery_pipeline/bronze_breweries"
        ),
        "quarantine": (
            "s3://lead-de-dev-bronze-data/"
            "brewery_pipeline/quarantined_breweries"
        ),
        "null_report": (
            "s3://lead-de-dev-bronze-data/"
            "brewery_pipeline/null_reports"
        ),
        "duplicate_ids": (
            "s3://lead-de-dev-bronze-data/"
            "brewery_pipeline/duplicate_ids"
        ),
        "curated": (
            "s3://lead-de-dev-silver-data/"
            "brewery_pipeline/curated_breweries"
        ),
        "gold": (
            "s3://lead-de-dev-gold-data/"
            "brewery_pipeline/gold_brewery_summary"
        ),
    }
```

tests/fixtures/breweries_valid.json

Repository: data-platform-jobs. Purpose: Known good API fixture

A single representative brewery record that allows reproducible local tests without calling the public API.

Current working-tree content:

```
[
  {
    "id": "test-brewery-001",
    "name": "Test Brewery",
    "brewery_type": "micro",
    "address_1": "1 Test Street",
    "address_2": null,
    "address_3": null,
    "city": "Test City",
    "state_province": "Test State",
    "postal_code": "12345",
    "country": "United States",
    "longitude": -73.9857,
    "latitude": 40.7484,
    "phone": "5551234567",
    "website_url": "https://example.com",
    "state": "Test State",
    "street": "1 Test Street"
  }
]
```

tests/fixtures/breweries_invalid.json

Repository: data-platform-jobs. Purpose: Known invalid API fixture

A representative record with a missing ID used to prove quarantine behaviour.

Current working-tree content:

```
[
  {
    "id": null,
    "name": "Test Brewery",
    "brewery_type": "micro",
    "address_1": "1 Test Street",
    "address_2": null,
    "address_3": null,
    "city": "Test City",
    "state_province": "Test State",
    "postal_code": "12 345",
    "country": "United States",
    "longitude": -73.9857,
    "latitude": 40.7484,
    "phone": "5 551 234 567",
    "website_url": "https://example.com",
    "state": "Test State",
    "street": "1 Test Street"
  }
]
```

.github/workflows/test.yml

Repository: data-platform-jobs. Purpose: Code quality and secure publisher workflow

Runs pull-request checks, builds a wheel, persists a temporary artifact, and on main pushes uses OIDC to publish the wheel and Glue entry script to a SHA-named dev prefix.

Current working-tree content:

```
name: Test Data Jobs

on:
  pull_request:
  push:
    branches:
      - main

permissions:
  contents: read
  id-token: write

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"

      - name: Install package and test dependencies
        run: python -m pip install -e ".[dev]"

      - name: Lint
        run: python -m ruff check .

      - name: Test
        run: python -m pytest -q

      - name: Build deployment artifacts
        run: python -m build

      - name: Upload deployment artifacts
        uses: actions/upload-artifact@v4
        with:
          name: glue-artifacts
          path: |
            dist/*.whl
            glue/glue_job.py
          if-no-files-found: error
          retention-days: 7

      - name: Configure AWS credentials for artifact publishing
        if: github.event_name == 'push' && github.ref == 'refs/heads/main'
        uses: aws-actions/configure-aws-credentials@v6.3.0
        with:
          role-to-assume: arn:aws:iam::930628638871:role/lead-de-dev-jobs-artifact-publisher
          aws-region: us-east-1
          allowed-account-ids: "930628638871"
          role-session-name: glue-artifact-publish-${{ github.run_id }}

      - name: Publish immutable Glue artifacts
        if: github.event_name == 'push' && github.ref == 'refs/heads/main'
        env:
          ARTIFACT_BUCKET: lead-de-dev-artifacts
        run: |
          set -euo pipefail

          artifact_prefix="s3://${ARTIFACT_BUCKET}/glue-artifacts/${GITHUB_SHA}"

          if aws s3 ls "${artifact_prefix}/" | grep -q .; then
            echo "Artifact path already exists: ${artifact_prefix}"
            exit 1
```

fi

```
aws s3 cp dist/*.whl "${artifact_prefix}/"  
aws s3 cp glue/glue_job.py "${artifact_prefix}/glue_job.py"  
aws s3 ls "${artifact_prefix}/"
```

.gitignore

Repository: data-platform-jobs. Purpose: Python workspace hygiene

Excludes virtual environments, caches, local data, build products, wheel metadata, and OS metadata from Git.

Current working-tree content:

```
.venv/  
__pycache__/  
.pytest_cache/  
*.pyc  
.env  
data/  
build/  
dist/  
*.egg-info/  
.DS_Store
```

Appendix C. Future Glue Terraform reference

This appendix is a proposed next change, not a claim that these files are already present. Review and adapt it before applying. It uses a dev-only Glue job because the artifact publisher and artifacts currently exist only for dev. It pins both the Git SHA and exact wheel name instead of looking up a moving latest artifact.

Additions to variables.tf

```
variable "glue_artifact_sha" {
  type      = string
  default    = null
  nullable   = true
  description = "Published Git SHA selected for the dev Glue job."
}

variable "glue_artifact_wheel_filename" {
  type      = string
  default    = null
  nullable   = true
  description = "Exact wheel filename in the selected Glue artifact prefix."
}
```

Both values default to null so the existing environments can continue to plan until a real artifact has been published. The job is enabled only when both values are present.

Additions to environments/dev.tfvars after a successful publish

```
environment = "dev"

# Copy these from a successful data-platform-jobs/main publishing run.
glue_artifact_sha      = "REPLACE_WITH_FULL_GIT_SHA"
glue_artifact_wheel_filename = "brewery_pipeline-0.1.0-py3-none-any.whl"
```

Do not commit a placeholder as a deployed input. Replace it only with an S3 prefix confirmed by the successful publish run. If the package version changes, use the actual filename printed by CI.

Proposed new file glue_job.tf

```
data "aws_caller_identity" "current" {}
data "aws_region" "current" {}

locals {
  brewery_glue_enabled = (
    var.environment == "dev" &&
    var.glue_artifact_sha != null &&
    var.glue_artifact_wheel_filename != null
  )

  brewery_artifact_prefix = "glue-artifacts/${var.glue_artifact_sha}"
  brewery_data_bucket_arns = [
    "arn:aws:s3:::lead-de-${var.environment}-bronze-data",
    "arn:aws:s3:::lead-de-${var.environment}-silver-data",
    "arn:aws:s3:::lead-de-${var.environment}-gold-data",
  ]
}

data "aws_iam_policy_document" "brewery_glue_assume_role" {
  statement {
    effect = "Allow"
    actions = ["sts:AssumeRole"]

    principals {
      type        = "Service"
      identifiers = ["glue.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "brewery_pipeline_glue" {
  count = local.brewery_glue_enabled ? 1 : 0
  name   = "lead-de-dev-brewery-pipeline-glue"
  assume_role_policy = data.aws_iam_policy_document.brewery_glue_assume_role.json
}
```

```

}

data "aws_iam_policy_document" "brewery_glue_s3" {
  count = local.brewery_glue_enabled ? 1 : 0

  statement {
    sid     = "ListSelectedArtifactAndGlueTempPrefixes"
    effect  = "Allow"
    actions = ["s3:ListBucket"]
    resources = [module.glue_artifacts.bucket_arn]

    condition {
      test     = "StringLike"
      variable = "s3:prefix"
      values = [
        "${local.brewery_artifact_prefix}/",
        "glue-temporary/"
      ]
    }
  }
}

statement {
  sid     = "ReadSelectedGlueArtifacts"
  effect  = "Allow"
  actions = ["s3:GetObject"]
  resources = [
    "${module.glue_artifacts.bucket_arn}/${local.brewery_artifact_prefix}/glue_job.py",
    "${module.glue_artifacts.bucket_arn}/${local.brewery_artifact_prefix}/${var.glue_artifact_wheel_filename}",
  ]
}

statement {
  sid     = "UseGlueTemporaryPrefix"
  effect  = "Allow"
  actions = [
    "s3:GetObject",
    "s3:PutObject",
    "s3:DeleteObject",
  ]
  resources = ["${module.glue_artifacts.bucket_arn}/glue-temporary/"]
}

statement {
  sid     = "ListDevelopmentDataBuckets"
  effect  = "Allow"
  actions = ["s3:ListBucket"]
  resources = local.brewery_data_bucket_arns
}

statement {
  sid     = "ReadWriteDevelopmentDeltaTables"
  effect  = "Allow"
  actions = [
    "s3:GetObject",
    "s3:PutObject",
    "s3:DeleteObject",
  ]
  resources = [for bucket_arn in local.brewery_data_bucket_arns : "${bucket_arn}/brewery_pipeline/"]
}
}

resource "aws_iam_role_policy" "brewery_glue_s3" {
  count = local.brewery_glue_enabled ? 1 : 0
  name  = "dev-brewery-pipeline-s3"
  role  = aws_iam_role.brewery_pipeline_glue[0].id
  policy = data.aws_iam_policy_document.brewery_glue_s3[0].json
}

data "aws_iam_policy_document" "brewery_glue_logs" {
  count = local.brewery_glue_enabled ? 1 : 0

  statement {
    sid     = "CreateGlueLogGroups"
    effect  = "Allow"
    actions = ["logs:CreateLogGroup"]
    resources = ["*"]
  }

  statement {
    sid     = "WriteGlueLogs"
    effect  = "Allow"
    actions = [
      "logs:CreateLogStream",
      "logs:PutLogEvents",
    ]
  }
}

```

```

resources = [
    "arn:aws:logs:${data.aws_region.current.name}:${data.aws_caller_identity.current.account_id}:log-group:/aws-glue/*:*",
]
}

resource "aws_iam_role_policy" "brewery_glue_logs" {
    count = local.brewery_glue_enabled ? 1 : 0
    name = "dev-brewery-pipeline-logs"
    role = aws_iam_role.brewery_pipeline_glue[0].id
    policy = data.aws_iam_policy_document.brewery_glue_logs[0].json
}

resource "aws_glue_job" "brewery_pipeline" {
    count = local.brewery_glue_enabled ? 1 : 0
    name = "lead-de-dev-brewery-pipeline"
    role_arn = aws_iam_role.brewery_pipeline_glue[0].arn
    glue_version = "5.1"
    worker_type = "G.1X"
    number_of_workers = 2
    max_retries = 0
    timeout = 30

    execution_property {
        max_concurrent_runs = 1
    }

    command {
        name = "gluectl"
        python_version = "3"
        script_location = "s3://${module.glue_artifacts.bucket_name}/${local.brewery_artifact_prefix}/glue_job.py"
    }

    default_arguments = {
        "--job-language" = "python"
        "--datalake-formats" = "delta"
        "--conf" = "spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension --conf spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCatalog --conf spark.delta.logStore.class=org.apache.spark.sql.delta.storage.S3SingleDriverLogStore"
        "--extra-py-files" = "s3://${module.glue_artifacts.bucket_name}/${local.brewery_artifact_prefix}/${var.glue_artifact_wheel_filename}"
        "--TempDir" = "s3://${module.glue_artifacts.bucket_name}/glue-temporary/brewery-pipeline/"
        "--environment" = var.environment
        "--page" = "1"
        "--per_page" = "20"
        "--batch_id" = "override-at-start-job-run"
        "--enable-metrics" = "true"
        "--enable-job-insights" = "true"
    }
}

output "dev_brewery_glue_job_name" {
    description = "Name of the dev Glue brewery job when an artifact is pinned."
    value = try(aws_glue_job.brewery_pipeline[0].name, null)
}

```

How to read the Glue reference

- The service trust allows Glue, not GitHub, to assume the execution role.
- The artifact permission names two full object locations. The job cannot read a different revision simply by changing a prefix at runtime.
- The data policy grants the Delta operations that the current code uses only beneath `brewery_pipeline/` in the development data buckets.
- The Glue temporary prefix is separate from `glue-artifacts/`. The publisher role still cannot touch it.
- The `batch_id` default is deliberately a guard string. Every real `StartJobRun` must override it with a meaningful stable run ID. Do not run the job repeatedly with the placeholder.
- The policy's `logs:CreateLogGroup` action has `Resource *` because CloudWatch Logs does not support narrowing that create action to a not-yet-existing log group. Review AWS service authorization before tightening it further.

Dependency packaging follow-up

The Glue reference adds the project wheel through `--extra-py-files` because that directly serves the package code. Before relying on it beyond a learning run, add a CI-built frozen dependency bundle or Glue virtual environment. If a wheel bundle is used, publish it in the same SHA directory and use `--additional-python-modules` with the bundle and `--python-modules-installer-option` set to `--no-index`. This prevents runtime PyPI resolution and makes requests and its transitive dependencies part of the reviewed artifact set.

Appendix D. Commands and source references

Useful local verification commands

```
# Infrastructure formatting and plan (run in the infrastructure repository)
terraform fmt -check -recursive
terraform init
terraform workspace select dev || terraform workspace new dev
terraform plan -var-file="environments/dev.tfvars"

# Data job local quality gate (run in data-platform-jobs)
python3.11 -m venv .venv
.venv/bin/python -m pip install -e ".[dev]"
JAVA_HOME=$(/usr/libexec/java_home -v 17) PATH="$JAVA_HOME/bin:$PATH" .venv/bin/python -m pytest -q
.venv/bin/python -m ruff check .
.venv/bin/python -m build

# Verify one published prefix after a successful main run
aws s3 ls s3://lead-de-dev-artifacts/glue-artifacts/REPLACE_WITH_FULL_GIT_SHA/
```

AWS and tool documentation consulted

AWS Glue runtime and Delta guidance: <https://docs.aws.amazon.com/glue/latest/dg/aws-glue-programming-etl-format-delta-lake.html>

AWS Glue job parameters: <https://docs.aws.amazon.com/glue/latest/dg/aws-glue-programming-etl-glue-arguments.html>

AWS Glue Python library and dependency guidance: <https://docs.aws.amazon.com/glue/latest/dg/aws-glue-programming-python-libraries.html>

AWS Glue job properties and runtime versions: <https://docs.aws.amazon.com/glue/latest/dg/add-job.html>

AWS GitHub credentials action OIDC quick start: <https://github.com/aws-actions/configure-aws-credentials>

Terraform AWS Glue job resource: https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/glue_job

Final operating rule

A successful CI build is not automatically deployed. It becomes deployable only when it is published into an immutable-looking SHA path. It becomes deployed only when a reviewed Terraform change pins an execution resource to that exact path. Keep those transitions separate, and you can always answer what code ran, who was allowed to publish it, and which AWS identity processed the data.